



UNIVERSITE HASSAN II DE CASABLANCA
FACULTE DES SCIENCES BEN M'SICK

Documentation PFE - PDF 01/10

Architecture Globale

Vue d'ensemble du systeme + Micro-services + Communication

Realise par :

AKRAM BELMOUSSA - Backend & Integration NLP

ZAKARIA - Frontend Angular & UX/UI

NOUHAILA - Base de donnees & Contenu FAQ

MAY 2026

Sommaire

Chapitre 1 - Qu'est-ce que la FSBM Platform ?	3
Chapitre 2 - Le probleme universitaire FSBM	6
Chapitre 3 - Objectifs du projet	9
Chapitre 4 - Concepts de base avant d'aller plus loin	12
Chapitre 5 - Architecture micro-services expliquee	18
Chapitre 6 - Les 6 services en detail	23
Chapitre 7 - Le frontend Angular (vue d'ensemble)	29
Chapitre 8 - Les bases de donnees (MySQL + MongoDB)	34
Chapitre 9 - Communication via HTTP/REST	40
Chapitre 10 - Les ports et leur utilite	45
Chapitre 11 - Workflow global d'une requete	49
Chapitre 12 - Choix techniques justifies	54
Chapitre 13 - Conclusion et lecture des autres PDFs	59

Chapitre 1 - Qu'est-ce que la FSBM Platform ?

1.1 Le projet en une phrase

FSBM Platform est une **plateforme universitaire intelligente** qui combine un **chatbot multilingue** (FR / EN / Darija marocaine) avec un **referentiel academique navigable** pour les etudiants de la Faculte des Sciences Ben M'Sick a Casablanca.

1.2 De quoi se compose le projet ?

Concretement, quand un etudiant ouvre la plateforme dans son navigateur, il voit :

Un dashboard d'accueil. Avec 8 cartes statistiques en temps reel (nombre d'etudiants, filieres, modules...), les dernieres annonces et les evenements a venir.

Une page Assistant IA. Un chatbot capable de repondre en 3 langues, avec memoire conversationnelle, personnalisation (khoya/khti selon le genre), et option LLaMA 3 pour des reponses plus naturelles.

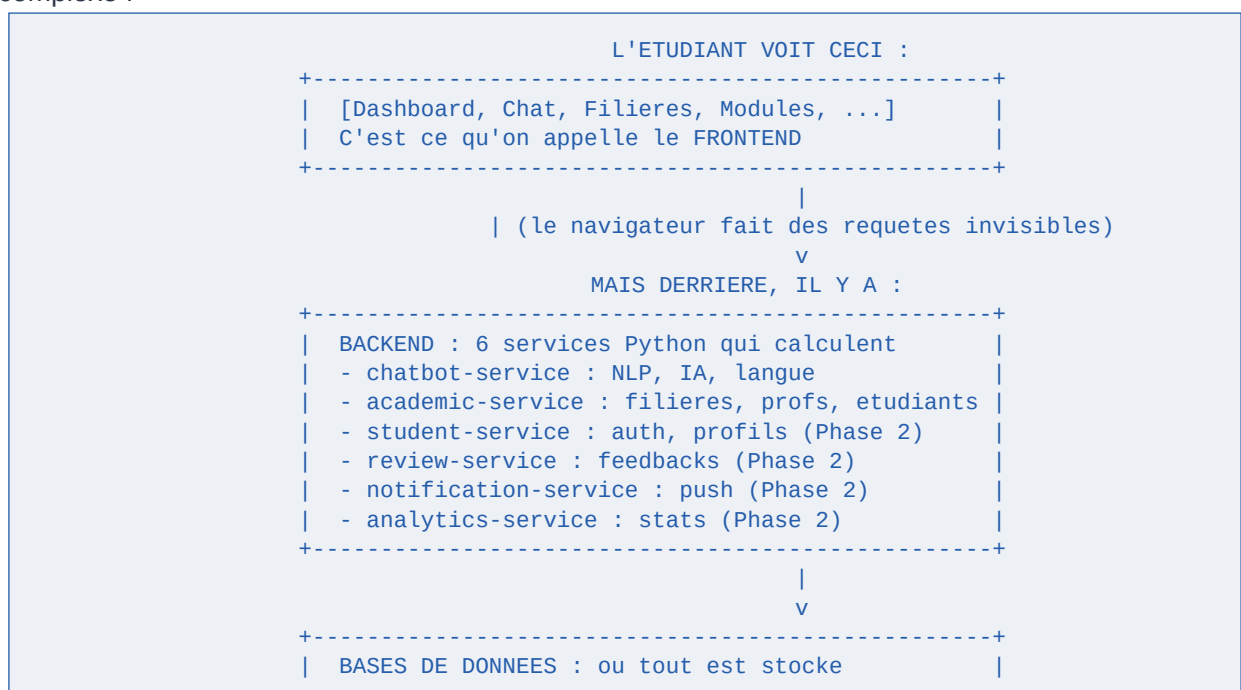
Des pages de navigation academique. 5 departements, 25 filieres (7 licences + 18 masters), 100+ modules, 107 professeurs, EDT, examens, annonces, clubs...

Un mode sombre. Toggle dans la sidebar, persiste entre sessions.

Un design responsive. Fonctionne sur mobile, tablette, desktop.

1.3 Qu'est-ce qu'il y a 'derriere' ce que voit l'utilisateur ?

L'utilisateur ne voit que la 'partie emergee de l'iceberg'. Derriere son ecran, il y a un systeme complexe :



```
| - MySQL : etudiants, filieres, modules |  
| - MongoDB : reviews, logs, sentiments |  
+-----+-----+
```

■ **ANALOGIE** : Imagine un **restaurant**. Le client (utilisateur) voit la salle (frontend Angular), passe commande au serveur (services backend FastAPI), le serveur va en cuisine (bases de donnees MySQL/MongoDB) chercher les ingredients, le chef cuisine (logique metier), et le serveur ramene le plat (la reponse). Le client ne voit jamais la cuisine, mais elle est essentielle au fonctionnement.

1.4 Pourquoi ce projet ?

Le projet est un **Projet de Fin d'Etudes (PFE)** de Licence Developpement Informatique a la FSBM. Il a 3 objectifs principaux :

Resoudre un vrai probleme universitaire. Les etudiants FSBM sont satures d'informations dispersees (site web, panneaux, scolarite, Facebook). Le chatbot centralise tout.

Demontrer la maitrise de technologies modernes. Angular 17, FastAPI, micro-services, NLP, LLM, RAG, MySQL+MongoDB... Le projet utilise les memes outils que les vraies entreprises tech (Microsoft, Google, startups marocaines).

Avoir un produit reellement deployable. Le code n'est pas un jouet : il est conçu pour pouvoir tourner en vrai dans la FSBM, avec donnees authentiques, scripts d'installation, documentation, etc.

Chapitre 2 - Le probleme universitaire FSBM

2.1 Le constat

Aujourd'hui a la FSBM, quand un etudiant veut savoir une information administrative (quand sont les examens ? comment s'inscrire au master IADS ? comment obtenir une attestation ?), il doit :

- Aller sur le site officiel www.fsbm.ma (souvent incomplet ou pas a jour)
- Consulter la page Facebook FSBMUH2C (annonces mais pas archivees, difficiles a chercher)
- Aller physiquement aux panneaux d'affichage de son departement
- Demander a la scolarite (file d'attente de 1-2 heures en periode de pointe)
- Demander a un camarade plus ancien (souvent l'info la plus fiable, ironiquement)
- Chercher dans des groupes WhatsApp non-officiels

Le probleme central

Resultat : **fragmentation** de l'information, perte de temps, inegalite d'accès (les etudiants bien connectes savent, les autres galèrent), saturation des services humains, et un sentiment general de 'la fac est mal organisee'.

2.2 Les 5 douleurs identifiees

Fragmentation des sources. Pour une question simple comme 'Quels documents pour s'inscrire ?', l'etudiant doit consulter 3 a 5 sources differentes. Aucune n'a toute l'information.

Saturation du service scolarite. En periode de pointe (rentree, examens, reinscription), le service scolarite recoit 200+ etudiants/jour avec des questions repetitives. 80% sont des questions de base qu'un chatbot peut absorber.

Indisponibilite hors horaires. Les services administratifs ferment a 16h30 et le weekend. Les etudiants qui travaillent ou habitent loin sont penalises - ils n'ont acces a l'info qu'aux memes horaires que tout le monde.

Inegalite d'accès a l'information. Les primo-entrants (1ere annee, premier semestre) n'ont pas de reseau d'anciens. Ils manquent des astuces que les anciens connaissent (quel prof est gentil, quels modules sont eliminatoires, comment reussir un concours...).

Absence d'outil moderne. La FSBM est en 2026 - une faculte de sciences ! - et ne dispose pas encore d'un point d'accès unifie a ses ressources academiques. Aucun moteur de recherche interne, aucun tableau de bord etudiant, aucun assistant intelligent.

2.3 Qui souffre de ces problemes ?

Trois categories d'acteurs sont impactes :

Les etudiants (3000+). Frustration, perte de temps, sentiment d'etre 'oublies'.

Le personnel administratif (~30 personnes). Surcharge de travail repetitif (repetier 100 fois par jour 'voila les documents'), stress, fatigue.

Le decanat. Image de marque ecornee, plaintes recurrentes des etudiants, difficulte a moderniser.

2.4 Pourquoi un chatbot ?

Un chatbot resout ces problemes pour 4 raisons :

Disponibilite 24h/24. Le chatbot ne dort jamais. Un etudiant qui revise a 23h peut poser sa question instantanement.

Reponses instantanees. Pas de file d'attente. La reponse arrive en moins d'1 seconde.

Multilingue. Notre chatbot comprend francais, anglais ET darija marocaine. Tres rare !

Centralisation. Toutes les sources sont agregees : site web, base de donnees academique, annonces officielles. L'etudiant n'a qu'un seul endroit a consulter.

Chapitre 3 - Objectifs du projet

3.1 Methode SMART

Les objectifs ont ete formules selon la methode SMART :

Lettre	Signification	Application dans notre projet
S	Specifique	Objectif clair, pas vague
M	Mesurable	Avec un indicateur chiffrable
A	Atteignable	Realiste avec nos ressources
R	Realiste	Pertinent et utile
T	Temporel	Avec une echeance

3.2 Les 7 objectifs concrets

Objectif 1 - Couvrir 60+ types de questions. Indicateur : nombre d'intents reconnus dans le dataset. Cible : 60+ intents. Resultat atteint : 28 intents enrichis (FR + EN + Darija) + RAG pour les variations.

Objectif 2 - Reponse en moins de 2 secondes. Indicateur : latence moyenne par requete. Cible : 2000ms. Resultat atteint : TF-IDF en 50-150ms, LLM en 150-300ms via Groq.

Objectif 3 - 3 langues supportees. Indicateur : langues du dataset et detection. Cible : FR + EN + Darija. Resultat atteint : 14/14 tests de detection langue passes.

Objectif 4 - Architecture professionnelle. Indicateur : architecture micro-services + bonnes pratiques. Cible : 6 services planifies. Resultat atteint : 2 livres, 4 en scaffolding.

Objectif 5 - Donnees realistes. Indicateur : volume du seed data. Cible : 3000+ etudiants. Resultat atteint : 107 profs + 2970 etudiants avec noms marocains.

Objectif 6 - Interface moderne. Indicateur : nombre de pages + features UX. Cible : 9 pages + mode sombre. Resultat atteint : 9 pages lazy-loaded, animations, responsive, dark mode.

Objectif 7 - Documentation exhaustive. Indicateur : nombre de PDFs. Cible : 3-5 PDFs. Resultat atteint : 6+ PDFs (rapport, guide tech, zombies, doc complete, guide IA, ces 10 PDFs).

3.3 Ce qui est HORS scope (Phase 2)

- Authentification reelle (JWT, login, sessions persistantes) - prevu Phase 2
- Reviews avec sentiment analysis - prevu Phase 2
- Notifications push en temps reel - prevu Phase 2
- Dashboard administrateur - prevu Phase 2
- Application mobile native - prevu Phase 3
- Integration avec Apogee (le SI universite) - hors scope (necessite accord institution)

Chapitre 4 - Concepts de base avant d'aller plus loin

Ce chapitre presente les concepts fondamentaux que tu DOIS comprendre avant de lire la suite. C'est volontairement tres vulgarise, comme pour un cours de 1ere annee.

4.1 C'est quoi un site web ?

Un **site web** est un ensemble de pages accessibles via un navigateur (Chrome, Firefox, Edge). Il y a 2 types historiques :

Sites statiques (annees 90)

Pages HTML pre-ecrites. Le serveur envoie un fichier HTML, le navigateur l'affiche. C'est tout. Exemple : un blog personnel des annees 2000.

Sites dynamiques avec rechargement (annees 2000)

Quand tu cliques sur un lien, le navigateur recharge toute la page depuis zero. Le serveur genere du HTML 'a la volee' avec PHP/JSP/ASP. Exemple : Wikipedia des annees 2010.

4.2 C'est quoi une SPA ?

SPA = Single Page Application = Application a Page Unique. C'est l'evolution moderne des sites web (annees 2015+). Le navigateur charge UNE SEULE page HTML au depart, puis utilise du JavaScript pour **modifier dynamiquement le contenu** sans jamais recharger la page entiere.

■ **ANALOGIE :** Imagine un **livre de coloriage**. Un site classique = chaque page est dans un livre different (rechargement complet). Une SPA = un seul livre, et tu effaces/redessine directement sur LA page selon ce que tu veux voir. C'est plus fluide et plus rapide.

Exemples connus de SPA : Gmail, Facebook, Twitter, Trello, Notion. Notre frontend Angular est une SPA.

Avantages de la SPA

- + Experience utilisateur fluide (pas de 'flash blanc' au changement de page)
- + Tres rapide une fois charge
- + Le serveur ne fait pas de generation HTML (moins de charge)
- + Le frontend et le backend peuvent etre developpes independamment

Inconvenients

- Premier chargement plus long (il faut telecharger tout le JavaScript)
- Moins bon pour le SEO (Google a du mal a indexer)
- Necessite du JavaScript active

4.3 C'est quoi Angular ?

Angular est un **framework JavaScript** developpe par Google. Il sert a creer des SPA de maniere structuree.

Un framework, c'est une **boite a outils complete** qui impose une organisation. Au lieu de coder tout from scratch, tu utilises les briques Angular (composants, services, routing, formulaires...) et tu te concentres sur ta logique metier.

■ **ANALOGIE** : Angular est comme un kit LEGO pre-assemble pour faire une voiture. Tu as deja les roues, le chassis, les portes. Tu construis le design et l'interieur. Sans framework, tu devrais fabriquer les roues toi-meme.

Notre projet utilise **Angular 17** (version sortie en novembre 2023) qui apporte :

- **Standalone components** : composants autonomes sans modules
- **Signals** : reactivite native (alternative a RxJS pour les cas simples)
- **Control flow** : nouvelle syntaxe `@if` `@for` plus claire
- **Lazy loading** : chargement a la demande des pages

4.4 C'est quoi un backend ?

Le **backend** (ou serveur, ou cote serveur) est la partie du systeme qui **n'est pas visible** par l'utilisateur. C'est le 'cerveau' qui :

- Recoit les requetes du frontend
- Verifie qui tu es (authentification)
- Cherche les donnees dans la base
- Calcule, applique la logique metier
- Renvoie les resultats au frontend

■ **ANALOGIE** : Le backend, c'est comme le **chef cuisinier** d'un restaurant. Le client (frontend) passe commande, le serveur (HTTP) transmet, le cuisinier (backend) prepare le plat avec les ingredients (base de donnees), et renvoie au client.

4.5 C'est quoi FastAPI ?

FastAPI est un framework Python pour creer des API REST. C'est l'equivalent d'Angular mais cote serveur, et en Python au lieu de TypeScript.

- **Rapide** : execution asynchrone native (async/await)
- **Validation automatique** : avec Pydantic (les donnees sont verifiees a l'arrivee)
- **Documentation auto** : Swagger UI genere automatiquement sur `/docs`
- **Moderne** : sortie en 2018, utilise les dernieres pratiques Python

4.6 C'est quoi une API ?

API = Application Programming Interface. C'est un **contrat** entre 2 logiciels. Quand le frontend veut parler au backend, il doit suivre les regles definies dans l'API.

■ **ANALOGIE :** L'API, c'est comme la **carte d'un restaurant**. Tu ne peux pas commander n'importe quoi - tu dois choisir parmi les plats listes, et les ingredients sont decrits. Une fois que tu commandes 'pizza margherita', tu sais ce que tu vas recevoir.

Notre backend expose une API REST avec des **endpoints** (= entrees, comme les plats de la carte) :

```
POST /api/chat          -> envoyer un message au chatbot
GET  /api/academic/filieres -> recuperer la liste des filieres
GET  /api/llm/status     -> verifier si LLaMA est dispo
```

4.7 C'est quoi REST ?

REST = Representational State Transfer. C'est un **style architectural** pour concevoir des API. Les regles :

URL = ressource. L'URL identifie une ressource (un objet). Ex: /api/filieres/SMI = la filiere SMI.

Methode HTTP = action. GET = lire, POST = creer, PUT = remplacer, DELETE = supprimer, PATCH = modifier partiellement.

Stateless. Chaque requete contient toute l'info necessaire. Le serveur ne se souvient pas des requetes precedentes.

Format standard. Generalement JSON. Plus rarement XML, YAML, etc.

4.8 C'est quoi JSON ?

JSON = JavaScript Object Notation. C'est un format de donnees structurees, lisible par humain et machine. C'est devenu le standard pour les API.

```
{
  "id": 1,
  "code": "SMI",
  "name": "Sciences Mathematiques et Informatique",
  "type": "LICENCE",
  "capacity": 180,
  "modules": [
    { "code": "M111", "name": "Analyse I" },
    { "code": "M112", "name": "Algebre I" }
  ],
  "is_active": true
}
```

Structure :

- Objets entre { }
- Tableaux entre []
- Cles entre guillemets

- Valeurs : string, nombre, booleen (true/false), null, autre objet, autre tableau

4.9 C'est quoi un micro-service ?

Un **micro-service** est une **application autonome** qui fait UNE chose et la fait bien. Au lieu d'avoir une grosse application qui fait tout (monolithe), on la decoupe en plusieurs petites applications qui communiquent entre elles via HTTP.

■ **ANALOGIE** : Le monolithe = un restaurant ou TOUS les employes font TOUT (caissier-serveur-cuisinier-vaisseliste). Si l'un est absent, tout s'arrete. Les micro-services = un restaurant ou chacun a son role : caissier, serveur, cuisinier, vaisseliste. Si le vaisseliste est absent, le restaurant continue (les autres assurent).

Avantages :

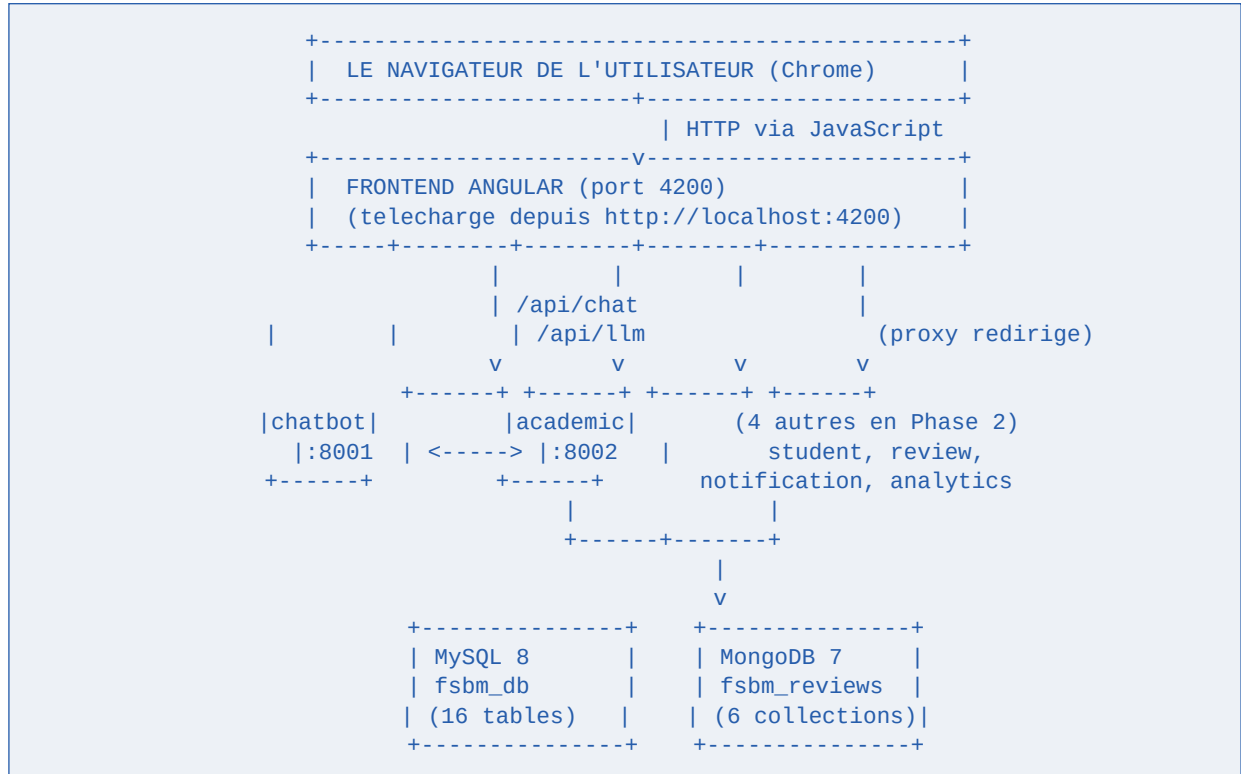
- + Independance : chaque service evolue separement
- + Resilience : si un service tombe, les autres continuent
- + Stack heterogene : un service en Python, un autre en Go, un autre en Rust...
- + Scalabilite : on peut multiplier seulement les services tres charges

Inconvenients :

- Complexite : il faut gerer le reseau, la latence, les pannes inter-services
- Debugging : plus dur de tracer une requete qui passe par plusieurs services
- Operations : il faut deployer N services au lieu d'1

Chapitre 5 - Architecture micro-services expliquee

5.1 Vue d'ensemble



5.2 Pourquoi cette architecture ?

Le projet aurait pu être un **monolithe** (tout en un seul gros service). On a choisi les micro-services pour 5 raisons stratégiques :

Demonstration pedagogique. Un PFE doit démontrer la maîtrise des pratiques modernes. Les grandes entreprises (Netflix, Uber, Amazon) utilisent toutes des micro-services. Le jury reconnaîtra que l'équipe sait construire un système moderne.

Separation des responsabilites. Chaque service a un rôle clair : chatbot fait du NLP, academic gère les données, student gère l'auth. C'est plus facile à comprendre et à maintenir.

Evolutivite independante. On peut ajouter un service (ex: notification) sans toucher aux autres. Phase 2 n'impacte pas Phase 1.

Stack heterogene. Le chatbot utilise scikit-learn (ML), l'academic utilise SQLAlchemy (ORM SQL), le review utiliserait Motor (driver MongoDB async). Chacun choisit ses outils.

Resilience. Si le service de reviews tombe, le chatbot et la navigation continuent de marcher. L'utilisateur n'est pas totalement bloqué.

5.3 Les 3 couches

L'architecture suit le pattern classique a 3 couches :

Couche 1 : Presentation (Frontend Angular)

Tout ce que l'utilisateur voit et avec quoi il interagit. C'est le HTML, CSS et TypeScript qui s'exécute dans son navigateur. **Aucune logique métier ici** - juste de l'affichage et des appels HTTP au backend.

Couche 2 : Services (Backend FastAPI)

La logique métier. Les services Python reçoivent les requêtes du frontend, calculent, interrogent les bases de données, et renvoient des réponses. **C'est ici qu'est l'intelligence.**

Couche 3 : Persistance (MySQL + MongoDB)

Le stockage permanent des données. Quand on coupe le serveur, les données restent. **Aucune logique ici** - juste du stockage structure et indexe.

[★] Couplage faible

Cette séparation stricte permet de remplacer une couche sans toucher aux autres. Par exemple, on pourrait remplacer Angular par React (couche 1) sans rien changer au backend.

Chapitre 6 - Les 6 services en detail

Le projet a planifie 6 micro-services. Phase 1 : 2 livres. Phase 2 : 4 a venir.

6.1 chatbot-service (port 8001) [LIVRE]

Mission : recevoir un message utilisateur et generer une reponse intelligente.

Sous-modules internes :

nlp/. Pipeline NLP : preprocessor (tokenisation, stopwords, stemming), language_detector (FR/EN/Darija), classifieur (TF-IDF + Cosine multi-langue).

llm/. Integration LLaMA 3 via Groq. groq_client.py, hf_client.py, rag.py, llm_service.py.

core/. Memory (session, historique), persona (genre/nom), web_fetcher (scraping fsbm.ma).

routers/. Endpoints FastAPI : /api/chat, /api/llm/chat, /api/intents, /api/health.

models/. Schemas Pydantic pour validation des inputs et outputs.

6.2 academic-service (port 8002) [LIVRE]

Mission : exposer les donnees academiques via API REST.

Endpoints :

- /api/overview - compteurs pour le dashboard
- /api/departments - 5 departements
- /api/filieres - 25 filieres (7 licences + 18 masters)
- /api/modules - 100+ modules
- /api/professors - annuaire paginé 107 profs
- /api/students - 2970 etudiants (filtres + pagination)
- /api/schedule - emploi du temps
- /api/exams - calendrier examens
- /api/announcements - annonces officielles
- /api/events - evenements universitaires
- /api/clubs - 8 clubs etudiants

6.3 student-service (Phase 2)

Mission : authentication JWT + gestion des profils + roles.

Endpoints prevus : POST /api/auth/login, POST /api/auth/register, GET /api/me, PUT /api/me/preferences, GET /api/users (admin only).

6.4 review-service (Phase 2)

Mission : CRUD des reviews MongoDB + sentiment analysis.

Endpoints prevus : POST /api/reviews, GET /api/reviews/by-category, POST /api/feedbacks/quick (pouce haut/bas), GET /api/sentiment/stats.

6.5 notification-service (Phase 2)

Mission : notifications push en temps reel via Server-Sent Events (SSE).

Endpoints prevus : GET /api/notifications/stream (SSE), POST /api/announcements (admin).

6.6 analytics-service (Phase 2)

Mission : dashboards administrateurs avec KPIs.

Endpoints prevus : GET /api/stats/overview, GET /api/stats/top-intents, GET /api/stats/satisfaction, GET /api/stats/usage-by-hour.

6.7 Pourquoi exactement 6 services ?

Le decoupage en 6 services suit le principe **Single Responsibility** : chaque service a UNE mission claire. Si on faisait 2 services seulement, on aurait des 'gros' services qui violeraient ce principe. Si on faisait 20 services, on aurait trop de complexite reseau. 6 est un bon equilibre pour un PFE.

Chapitre 7 - Le frontend Angular (vue d'ensemble)

Ce chapitre est volontairement bref - le PDF 02 detaille Angular en profondeur.

7.1 Structure

```
frontend/
  src/
    main.ts           -> point d'entree, demarre l'app
    index.html        -> page HTML unique (la 'P' de SPA)
    styles.css         -> styles globaux + variables theme
    assets/           -> images, logos, ressources
    app/
      app.component.ts -> composant racine
      app.routes.ts    -> definition des 9 routes
      layout/          -> sidebar + topbar shared
      core/            -> theme service, etc.
      services/        -> chat.service, academic.service
      features/        -> 9 pages lazy-loaded
      components/      -> chat-window, message-bubble, etc.
      models/          -> interfaces TypeScript
```

7.2 Les 9 pages

Route	Composant	Role
/	DashboardComponent	Accueil avec stats
/chat	ChatPageComponent	Assistant IA
/departements	DepartmentsComponent	5 cartes departements
/filieres	FilieresComponent	25 cartes filieres
/filieres/:code	FiliereDetailComponent	Programme par semestre
/modules	ModulesComponent	100+ modules filtrables
/professeurs	ProfessorsComponent	Annuaire profs paginé
/actualites	NewsComponent	Annonces + events
/vie-etudiante	StudentLifeComponent	8 clubs etudiants

7.3 Communication avec le backend

Le frontend ne 'parle' jamais directement aux services. Il passe par le **proxy Angular dev server** qui redirige selon les regles de proxy.conf.json :

```
// proxy.conf.json
{
  "/api/chat":      { "target": "http://localhost:8001" },
  "/api/llm":       { "target": "http://localhost:8001" },
  "/api/academic":  { "target": "http://localhost:8002",
```



```
    "pathRewrite": {"^/api/academic": "/api"} }  
  }
```

Ainsi, du point de vue Angular, tout passe par '/api/...' (URL relatives). Le proxy redirige automatiquement vers le bon micro-service.

Chapitre 8 - Les bases de donnees

8.1 Pourquoi 2 bases ?

Le projet utilise **MySQL ET MongoDB** simultanement. Ce n'est pas pour faire compliqué - c'est une pratique appelee **polyglot persistence** : chaque type de donnees est stocke dans la BDD qui lui convient le mieux.

Type de donnees	Caracteristique	BDD ideale
Etudiants, filieres, modules	Tres lies entre eux, schema fixe	MySQL (relationnel)
Reviews textuelles	Schema variable, gros volume	MongoDB (documentaire)
Logs d'usage	Ecritures massives, peu de lectures	MongoDB
Notes d'examen	Liaisons fortes etudiant-module	MySQL
Sentiments analyses	Structure JSON imbriquee	MongoDB
Annonces officielles	Structure stable, peu d'evolution	MySQL

8.2 MySQL - tour d'horizon

MySQL est une base de donnees **relationnelle** (tableaux relies entre eux par des cles etrangeres). Notre base s'appelle fsbm_db et contient **16 tables** :

- departments (5 rows) - les 5 departements
- filieres (25 rows) - 7 licences + 18 masters
- modules (100+ rows) - matieres par filiere
- professors (107 rows) - enseignants
- students (2970 rows) - etudiants inscrits
- module_teachers - relation N:N module/prof
- schedules - emploi du temps
- exams - calendrier examens
- grades (9000+ rows) - notes
- faq_categories - 16 categories
- faq_items - items FAQ
- conversations - historique chatbot

- feedbacks - notes utilisateurs
- announcements - 5+ annonces
- events - 5+ evenements
- clubs - 8 clubs

Pour le detail des relations entre tables, voir le PDF 04 - MySQL + SQLAlchemy.

8.3 MongoDB - tour d'horizon

MongoDB est une base de donnees **documentaire** (chaque entree est un document JSON). Notre base s'appelle fsbm_reviews et contient **6 collections** :

- reviews - avis textuels detaillés
- chatbot_feedback - pouce haut/bas
- conversations - log complet sessions
- sentiment_analysis - sentiments analyses
- usage_logs - tracking anonyme
- suggestions - suggestions ameliorations

Pour le detail, voir le PDF 05 - MongoDB + NoSQL.

Chapitre 9 - Communication via HTTP/REST

9.1 C'est quoi HTTP ?

HTTP = HyperText Transfer Protocol. C'est le **langage** que parlent les navigateurs et les serveurs web depuis 1991. Chaque fois que tu visites un site web, ton navigateur fait une requete HTTP.

9.2 Anatomie d'une requete HTTP

```
REQUETE :
POST /api/chat HTTP/1.1
Host: localhost:8001
Content-Type: application/json
Authorization: Bearer eyJ0eXAi...

{
  "message": "Quelles sont les filieres ?",
  "session_id": "sess_abc123"
}

REPONSE :
HTTP/1.1 200 OK
Content-Type: application/json

{
  "response": "La FSBM propose 7 licences...",
  "intent": "filieres",
  "confidence": 1.0,
  "session_id": "sess_abc123"
}
```

9.3 Les 5 methodes HTTP principales

Methode	Quand l'utiliser	Exemple FSBM
GET	Lire une ressource	GET /api/filieres
POST	Creer une ressource	POST /api/chat
PUT	Remplacer entierement	PUT /api/me/preferences
PATCH	Modifier partiellement	PATCH /api/users/42
DELETE	Supprimer	DELETE /api/reviews/123

9.4 Les codes de statut HTTP

Code	Categorie	Signification
200	Succes	OK - tout va bien
201	Succes	Created - ressource creee

204	Succes	No Content - succes sans contenu retourne
301	Redirection	Permanent Redirect
400	Erreur client	Bad Request - mauvais format
401	Erreur client	Unauthorized - pas authentifie
403	Erreur client	Forbidden - pas les droits
404	Erreur client	Not Found - ressource introuvable
422	Erreur client	Unprocessable Entity - validation echouee
429	Erreur client	Too Many Requests - rate limit
500	Erreur serveur	Internal Server Error - bug serveur
502	Erreur serveur	Bad Gateway - service en aval down
503	Erreur serveur	Service Unavailable - service down

9.5 Communication frontend -> backend

```

[User clique 'Envoyer message']
    |
    v
[Angular ChatService.sendMessage()]
    |
    | this.http.post('/api/chat', { message, session_id })
    v
[Navigateur fait requete HTTP POST]
    |
    | URL : http://localhost:4200/api/chat
    v
[Angular Dev Proxy redirige]
    |
    | URL reecrite : http://localhost:8001/api/chat
    v
[FastAPI recoit la requete]
    |
    | Valide avec Pydantic, traite, repond
    v
[Reponse JSON]
    |
    | { response, intent, confidence, ... }
    v
[Angular recoit dans subscribe()]
    |
    v
[Affichage dans MessageBubble]

```

Chapitre 10 - Les ports et leur utilite

10.1 C'est quoi un port ?

Un **port** est un nombre entre 0 et 65535 qui identifie une application sur un ordinateur. Quand on dit 'localhost:8001', '8001' est le port.

■ **ANALOGIE** : Imagine que ton ordinateur est un **immeuble** et que ton adresse IP (localhost = 127.0.0.1) est l'**adresse postale** de l'immeuble. Le port est le **numero d'appartement**. Pour livrer un colis (paquet HTTP) au bon destinataire (application), il faut le bon numero d'appartement.

10.2 Les ports utilises dans FSBM Platform

Port	Service	Role
4200	Angular dev server	Sert les fichiers HTML/CSS/JS du frontend
8001	chatbot-service	NLP, chatbot, LLM
8002	academic-service	Donnees academiques (filieres, profs...)
3306	MySQL Server	BDD relationnelle (par default)
27017	MongoDB Server	BDD documentaire (par default)

10.3 Pourquoi 8001 et 8002 et pas 5001/5002 ?

Initialement, on utilisait 5001 (chatbot) et 5002 (academic). Mais sous Windows + WSL2, ces ports sont parfois **squattes par des processus zombies** du subsystem Linux. On a migre vers 8001/8002 qui sont plus loin de la plage des reservations Hyper-V.

Pour les details du probleme des ports zombies et de la solution, voir le PDF FSBM_Platform_Guide_Zombies.pdf.

10.4 Comment changer un port ?

```
# Pour academic-service
# Editer services/academic-service/.env :
SERVICE_PORT=8002

# Ou ligne de commande directe :
py -m uvicorn app.main:app --port 8002

# Pour Angular
# Editer frontend/proxy.conf.json :
{
  "/api/chat": { "target": "http://localhost:8001" }
}
```

Chapitre 11 - Workflow global d'une requete

Voici ce qui se passe quand un etudiant tape un message dans le chatbot. Note les temps approximatifs - tout cela se passe en moins de 300ms perçus comme instantanes.

- T+0 ms** - L'utilisateur clique 'Envoyer' dans son navigateur.
- T+5 ms** - Angular intercepte le clic, le `ChatService.sendMessage()` est appele.
- T+10 ms** - Angular construit une requete HTTP POST avec body JSON.
- T+15 ms** - Le navigateur envoie : POST `http://localhost:4200/api/chat`
- T+20 ms** - Le dev server Angular consulte `proxy.conf.json`
- T+25 ms** - Il reecrit l'URL : `http://localhost:8001/api/chat`
- T+30 ms** - Le paquet TCP arrive sur `localhost:8001`
- T+35 ms** - `uvicorn` (serveur ASGI) recoit la requete, la passe a FastAPI
- T+40 ms** - FastAPI deserialise le JSON via Pydantic `ChatRequest`
- T+45 ms** - Validation : message length, session_id format, etc.
- T+50 ms** - Le router `chat.py` prend la main : `chat()` est appele
- T+55 ms** - `memory.get_or_create()` recupere ou cree la session
- T+60 ms** - `detect_gender()`, `detect_name()` scannent le message
- T+65 ms** - `classifier.predict()` lance le pipeline NLP
- T+70 ms** - `preprocessor.preprocess()` : lowercase, no stopwords, stemming
- T+75 ms** - `language_detector` detecte la langue (FR/EN/Darija)
- T+80 ms** - `vectorizer.transform()` vectorise le message
- T+90 ms** - `cosine_similarity()` calcule contre 461 patterns Darija
- T+100 ms** - Top intent identifie + reponse pre-ecrite recuperee
- T+110 ms** - `personalize_response()` substitue {voc} -> 'khoya' / 'khti'
- T+120 ms** - `save_conversation()` persiste en MySQL (asynchrone)
- T+130 ms** - `memory.add_turn()` enrichit la session en RAM
- T+140 ms** - `ChatResponse` construite avec metadata
- T+150 ms** - FastAPI serialise en JSON via Pydantic
- T+155 ms** - `uvicorn` envoie la reponse HTTP 200
- T+160 ms** - Le paquet revient sur `localhost:4200`

T+165 ms - Angular subscribe() est notifie de la reponse

T+170 ms - Le composant push le message dans messages[]

T+200 ms - Angular detecte le changement, met a jour le DOM

T+250 ms - Animation fadeIn du message bubble

T+300 ms - L'utilisateur voit la reponse, perception 'instantanee'

[✓] Tout ce parcours se passe en moins d'**1/3 de seconde**. La complexite est masquee par la rapidite du systeme.

Chapitre 12 - Choix techniques justifies

Le jury va POSER ces questions. Voici les reponses preparees.

Q : Pourquoi Angular plutot que React ou Vue ?

R : Angular est plus opinionated (impose une structure), ce qui force de bonnes pratiques. Il integre nativement le routing, les formulaires, l'HTTP, les animations. Pour un projet PFE, c'est plus simple a structurer. React est plus flexible mais demande plus de choix.

Q : Pourquoi FastAPI plutot que Flask/Django ?

R : FastAPI est async natif (plus performant), valide automatiquement les inputs avec Pydantic, et genere Swagger UI gratuitement. Flask demande des libs externes pour tout ca. Django est trop monolithique pour des micro-services.

Q : Pourquoi MySQL + MongoDB et pas une seule BDD ?

R : Polyglot persistence : on choisit la BDD la mieux adaptee a chaque type de donnees. MySQL pour les donnees relationnelles (etudiants, filieres), MongoDB pour les donnees flexibles (reviews, logs).

Q : Pourquoi pas Docker ?

R : Le cahier des charges PFE impose une compatibilite Windows sans conteneurs. Docker Desktop est lourd, conflits Hyper-V courants. On utilise des scripts batch/PowerShell et venv Python a la place.

Q : Pourquoi TF-IDF et pas un LLM directement ?

R : TF-IDF est gratuit, deterministe, explicable, ultra-rapide (5ms). Le LLM (LLaMA via Groq) est en option pour les questions complexes. Le TF-IDF sert aussi de retrieval pour le RAG du LLM.

Q : Pourquoi 6 services et pas 1 ?

R : Separation des responsabilites + demonstration des patterns modernes. Chaque service fait UNE chose et peut evoluer independamment.

Q : Comment gerer la securite (Phase 2) ?

R : JWT signe HS256 + bcrypt 12 rounds + 4 roles (STUDENT/PROF/SCOLARITE/ADMIN) + CORS strict + validation Pydantic + rate limiting.

Q : Comment scaler si 10 000 utilisateurs ?

R : Replication horizontale de chaque service + cache Redis + MySQL master/slave + MongoDB replica set + load balancer en front.

Chapitre 13 - Conclusion et lecture des autres PDFs

13.1 Ce que tu as appris ici

- + Le contexte FSBM et le probleme resolu
- + L'architecture micro-services en 3 couches
- + Les 6 services et leur role
- + Les 9 pages du frontend Angular
- + Les 2 bases de donnees (MySQL + MongoDB) et leur complementarite
- + Le protocole HTTP, REST, JSON
- + Les ports et leur utilite
- + Le workflow complet d'une requete (T+0 a T+300 ms)
- + Les 8 choix techniques justifies

13.2 Ou aller maintenant ?

Ce PDF est volontairement **generaliste**. Pour aller en profondeur sur chaque aspect, consulte les 9 autres PDFs :

PDF 02 - Frontend Angular. Angular 17 ligne par ligne, signals, components, services, etc.

PDF 03 - Backend FastAPI. FastAPI complet, async, Pydantic, routers, code expliqué

PDF 04 - MySQL + SQLAlchemy. SQL complet, 16 tables, relations, ORM, normalisation

PDF 05 - MongoDB + NoSQL. NoSQL, 6 collections, Motor async, comparaison SQL/NoSQL

PDF 06 - NLP + IA chatbot. Pipeline NLP complet, TF-IDF, RAG, LLM, embeddings

PDF 07 - APIs + Communication. REST détaillé, tous endpoints, communication inter-services

PDF 08 - Securite. JWT, bcrypt, OWASP, validation, CORS, roles

PDF 09 - Workflow complet. Step-by-step, 4 scenarios, tous les details

PDF 10 - Guide soutenance. 30+ Q/R, demo script, vulgarisation, backup plan

13.3 Ordre de lecture recommande

Ordre	PDF	Pourquoi maintenant
1	PDF 01 (ce document)	Vue d'ensemble
2	PDF 10 - Soutenance	Pour savoir ce que le jury veut entendre
3	PDF 09 - Workflow	Pour comprendre le flux global

4	PDF 02 - Frontend	Approfondir Angular
5	PDF 03 - Backend	Approfondir FastAPI
6	PDF 06 - NLP/IA	Comprendre le cerveau du chatbot
7	PDF 04 - MySQL	Approfondir la BDD principale
8	PDF 05 - MongoDB	Approfondir le NoSQL
9	PDF 07 - APIs	Reference des endpoints
10	PDF 08 - Securite	Pour les questions securite

[■] Conseil critique

Si tu ne dois lire qu'**UN SEUL** autre PDF apres celui-ci, lis le **PDF 10 - Guide soutenance**. C'est celui qui te prepare le mieux a l'examen oral.